

2

◆ Goal: Test various data types and typecasting in Java

1. Data Types in Java:

- **Primitive types:** `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`
- **Non-primitive types:** `String`, arrays, objects, etc.

2. Typecasting:

- **Implicit Casting (Widening):** Automatically converting smaller data type to bigger data type.
 - Example: `int` to `long`, `float` to `double`.
- **Explicit Casting (Narrowing):** Manually converting bigger data type to smaller data type.
 - Example: `double` to `int`.

Java Program Demonstrating Data Types and Typecasting

```
public class DataTypesTypeCasting {  
    public static void main(String[] args) {  
        // 1. Primitive Data Types  
        byte b = 10;  
        short s = 100;  
        int i = 1000;  
        long l = 10000L;  
        float f = 12.5f;  
        double d = 123.456;  
        char c = 'A';  
        boolean bool = true;  
  
        System.out.println("Byte value: " + b);  
        System.out.println("Short value: " + s);  
        System.out.println("Int value: " + i);  
        System.out.println("Long value: " + l);  
        System.out.println("Float value: " + f);  
        System.out.println("Double value: " + d);  
        System.out.println("Char value: " + c);  
        System.out.println("Boolean value: " + bool);  
    }  
}
```

// 2. Typecasting Examples

// Implicit Casting (Widening)

```
int i2 = b;    // byte to int
```

```
long l2 = i;   // int to long
```

```
double d2 = f; // float to double
```

```
System.out.println("\nImplicit Casting:");
```

```
System.out.println("byte to int: " + i2);
```

```
System.out.println("int to long: " + l2);
```

```
System.out.println("float to double: " + d2);
```

// Explicit Casting (Narrowing)

```
int i3 = (int) d;    // double to int
```

```
byte b3 = (byte) i;  // int to byte (possible data loss)
```

```
char c3 = (char) i;  // int to char
```

```
System.out.println("\nExplicit Casting:");
```

```
System.out.println("double to int: " + i3);
```

```
System.out.println("int to byte: " + b3);
```

```
System.out.println("int to char: " + c3);
```

```
// Typecasting char to int
```

```
int i4 = c;
```

```
System.out.println("\nChar to int (ASCII value): " + i4);
```

```
// Casting between numeric and char
```

```
char c4 = 66; // ASCII 66 = 'B'
```

```
System.out.println("int 66 cast to char: " + c4);
```

```
}
```

```
}
```

Output:

```
Byte value: 10
Short value: 100
Int value: 1000
Long value: 10000
Float value: 12.5
Double value: 123.456
Char value: A
Boolean value: true
```

Implicit Casting:

```
byte to int: 10
int to long: 1000
float to double: 12.5
```

Explicit Casting:

```
double to int: 123
int to byte: -24
int to char: 2
```

```
Char to int (ASCII value): 65
```

```
int 66 cast to char: B
```

- **Implicit casting** happens automatically when assigning smaller to bigger types.
- **Explicit casting** needs a cast operator `(type)` and may cause **loss of data** or unexpected results (like `int to byte` resulting in `-24` due to overflow).
- `char` stores ASCII values internally; casting between `char` and `int` uses ASCII codes.



Problem 1: Rectangle Class with Overloaded Constructors

In Java, **constructor overloading** means having more than one constructor in a class with different parameter lists. It allows you to create objects in different ways.

```
class Rectangle {  
    int length, breadth;  
  
    // Constructor with two parameters  
    Rectangle(int l, int b) {  
        length = l;  
        breadth = b;  
    }  
  
    // Constructor with one parameter (square)  
    Rectangle(int side) {  
        length = breadth = side;  
    }  
  
    void displayArea() {  
        System.out.println("Area: " + (length * breadth));  
    }  
}
```

```
public class TestRectangle {  
    public static void main(String[] args) {  
        Rectangle rect1 = new Rectangle(10, 5); // rectangle  
        Rectangle rect2 = new Rectangle(7);      // square  
  
        System.out.println("Rectangle:");  
        rect1.displayArea();  
  
        System.out.println("Square:");  
        rect2.displayArea();  
    }  
}
```

Output:

```
Rectangle:  
Area: 50  
Square:  
Area: 49
```


Problem 2: Employee Class with Overloaded Constructors

```
class Employee {  
    String name;  
    double salary;  
    String department;  
  
    // Constructor 1: name and salary  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
        this.department = "General"; // default  
    }  
  
    // Constructor 2: name, salary, and department  
    Employee(String name, double salary, String department) {  
        this.name = name;  
        this.salary = salary;  
        this.department = department;  
    }  
}
```

```
void display() {  
    System.out.println("Name: " + name + ", Salary: " + salary + ", Department: " + department);  
}  
  
public class TestEmployee {  
    public static void main(String[] args) {  
        Employee emp1 = new Employee("Alice", 50000);  
        Employee emp2 = new Employee("Bob", 60000, "IT");  
  
        emp1.display();  
        emp2.display();  
    }  
}
```

Output:

```
Name: Alice, Salary: 50000.0, Department: General  
Name: Bob, Salary: 60000.0, Department: IT
```

4

Array of Objects in Java

Problem 1: Store and Display Student Information

An *array of objects* in Java is simply an array where each element is an object of a class.

```
class Student {  
    String name;  
    int rollNo;  
    double marks;  
  
    Student(String name, int rollNo, double marks) {  
        this.name = name;  
        this.rollNo = rollNo;  
        this.marks = marks;  
    }  
  
    void display() {  
        System.out.println("Name: " + name + ", Roll No: " + rollNo + ", Marks: " + marks);  
    }  
}
```

```
public class StudentArray {  
    public static void main(String[] args) {  
        Student[] students = new Student[5];  
        students[0] = new Student("Alice", 101, 89.5);  
        students[1] = new Student("Bob", 102, 76.3);  
        students[2] = new Student("Charlie", 103, 91.2);  
        students[3] = new Student("David", 104, 67.8);  
        students[4] = new Student("Eva", 105, 85.6);  
  
        for (Student s : students) {  
            s.display();  
        }  
    }  
}
```

Output:

```
Name: Alice, Roll No: 101, Marks: 89.5  
Name: Bob, Roll No: 102, Marks: 76.3  
Name: Charlie, Roll No: 103, Marks: 91.2  
Name: David, Roll No: 104, Marks: 67.8  
Name: Eva, Roll No: 105, Marks: 85.6
```

Problem 2: Book Library System

```
class Book {  
    String title;  
    String author;  
    double price;  
  
    Book(String title, String author, double price) {  
        this.title = title;  
        this.author = author;  
        this.price = price;  
    }  
  
    void display() {  
        System.out.println("Title: " + title + ", Author: " + author + ", Price: $" + price);  
    }  
}
```

```
public class BookArray {  
    public static void main(String[] args) {  
        Book[] books = {  
            new Book("The Alchemist", "Paulo Coelho", 300.50),  
            new Book("1984", "George Orwell", 350.00),  
            new Book("Java Complete Ref", "Herbert Schildt", 700.00)  
        };  
  
        for (Book b : books) {  
            b.display();  
        }  
    }  
}
```

Output:

```
Title: The Alchemist, Author: Paulo Coelho, Price: $300.5  
Title: 1984, Author: George Orwell, Price: $350.0  
Title: Java Complete Ref, Author: Herbert Schildt, Price: $700.0
```



Command Line Arguments in Java

Command line arguments are passed when the Java program is executed from the terminal.

Problem 1: Sum of Two Numbers from Command Line

```
public class SumArgs {  
    public static void main(String[] args) {  
        if (args.length < 2) {  
            System.out.println("Please provide 2 numbers as arguments.");  
            return;  
        }  
  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
        System.out.println("Sum: " + (a + b));  
    }  
}
```

```
java SumArgs 10 20
```

```
public class GreetUser {  
    public static void main(String[] args) {  
        if (args.length < 1) {  
            System.out.println("Please provide your name as an argument.");  
            return;  
        }  
  
        String name = args[0];  
        System.out.println("Hello, " + name + "! Welcome to Java.");  
    }  
}
```

```
java GreetUser Alice
```


6

Inheritance in Java

Inheritance allows one class to **acquire the properties and methods** of another class. This promotes **code reusability** and helps model real-world relationships.

Problem 1: Single Inheritance

Output:

```
This animal eats food.  
The dog barks.
```

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("The dog barks.");  
    }  
}  
  
public class SingleInheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();    // Inherited method  
        d.bark();   // Own method  
    }  
}
```

Problem 2: Multi-level Inheritance

A class inherits from a class that itself inherits from another class.

```
class Vehicle {  
    void start() {  
        System.out.println("Vehicle starts.");  
    }  
}  
  
class Car extends Vehicle {  
    void drive() {  
        System.out.println("Car is driving.");  
    }  
}  
  
class SportsCar extends Car {  
    void turboBoost() {  
        System.out.println("SportsCar activates turbo boost!");  
    }  
}
```

```
public class MultilevelInheritance {  
    public static void main(String[] args) {  
        SportsCar sc = new SportsCar();  
        sc.start();           // Inherited from Vehicle  
        sc.drive();           // Inherited from Car  
        sc.turboBoost();      // Defined in SportsCar  
    }  
}
```

Output:

```
Vehicle starts.  
Car is driving.  
SportsCar activates turbo boost!
```

7

Dynamic Method Dispatch (Runtime Polymorphism)

- When a superclass reference refers to a **subclass object**.
- The **overridden method** that gets executed is based on the **object type**, not the reference type.
- This is resolved at **runtime**, hence called *dynamic*.

```
class Shape {  
    void draw() {  
        System.out.println("Drawing a shape.");  
    }  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing a circle.");  
    }  
}  
  
class Rectangle extends Shape {  
    void draw() {  
        System.out.println("Drawing a rectangle.");  
    }  
}
```

```
public class DynamicDispatch {  
    public static void main(String[] args) {  
        Shape s; // Superclass reference  
  
        s = new Circle(); // Subclass object  
        s.draw(); // Calls Circle's draw()  
  
        s = new Rectangle(); // New subclass object  
        s.draw(); // Calls Rectangle's draw()  
    }  
}
```

Output:

Drawing a circle.

Drawing a rectangle.



Creating Packages with Access Control

- A **package** is a group of related classes.
- Packages help in **organizing code**, **avoiding name conflicts**, and **controlling access** using modifiers (`public`, `private`, `protected`, default).
- You can **import** one package into another class using `import`.

✓ **Problem:** Create a package `mypack` containing a class `Person`. Use access modifiers and import it in another class.

1. 📁 **File:** `mypack/Person.java`

java

```
package mypack;
```

```
public class Person {
```

```
    private String name;
```

```
    protected int age;
```

```
    public Person(String name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    }
```

```
    public void display() {
```

```
        System.out.println("Name: " + name + ", Age: " + age);
```

```
    }
```

```
}
```

2. 📁 File: TestPerson.java

```
java

import mypack.Person;

public class TestPerson {
    public static void main(String[] args) {
        Person p = new Person("Alice", 25);
        p.display();
        // System.out.println(p.name); // Error: 'name' has private access
        // System.out.println(p.age);  // Error if not in same package
    }
}
```

Output:

Name: Alice, Age: 25

- The `Person` class is part of the `mypack` package.
- It is imported using `import mypack.Person;`
- `name` is `private` → cannot be accessed outside the class.
- `age` is `protected` → accessible in the same package or by subclass.
- `display()` is `public` → accessible everywhere.

9

User-Defined Exception Class

- You can create your own exceptions by **extending** `Exception` (for checked) or `RuntimeException` (for unchecked).
- Throw them using `throw` keyword.
- Use `try-catch` blocks to handle them.

✓ **Problem:** Create a custom exception `InvalidAgeException` which is thrown if `age < 18`.

📁 **File:** `InvalidAgeException.java`

java

```
class InvalidAgeException extends Exception {  
    public InvalidAgeException(String message) {  
        super(message);  
    }  
}
```

java

```
import java.util.Scanner;

public class TestCustomException {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        try {
            if (age < 18) {
                throw new InvalidAgeException("You must be at least 18 years old.");
            } else {
                System.out.println("Access granted. You are eligible.");
            }
        } catch (InvalidAgeException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

Enter your age: 20
Access granted. You are eligible.

Enter your age: 16
Exception caught: You must be at least 18 years old.

- `InvalidAgeException` is a custom checked exception.
- It's **thrown manually** using `throw`.
- It's **handled** using a `try-catch` block in the main method.
- This is useful for adding custom logic in real-world applications like form validation, business rules, etc.

10

Achieve Synchronization in Multithreading

- In multithreaded applications, **multiple threads can access shared resources**, which may lead to **data inconsistency**.
- **Synchronization** ensures that **only one thread at a time** can access a **critical section** (shared code/resource).

You can achieve synchronization using:

- `synchronized` **methods**
- `synchronized` **blocks**

✓ Problem: **Banking System Simulation**

◆ **Description:**

Simulate a bank account where multiple threads (e.g., ATM and online banking) are trying to **withdraw money at the same time** from the same account.

We'll synchronize the withdrawal process to avoid inconsistency.

```
class Bank {  
    private int balance = 1000;  
  
    // synchronized method to ensure only one thread can access it at a time  
    synchronized void withdraw(int amount, String threadName) {  
        if (balance >= amount) {  
            System.out.println(threadName + " is going to withdraw ₹" + amount);  
            try {  
                Thread.sleep(100); // simulate time delay  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            balance -= amount;  
            System.out.println(threadName + " has completed withdrawal. Remaining Balance: ₹" + balance);  
        } else {  
            System.out.println(threadName + " tried to withdraw ₹" + amount + " but insufficient balance. Remaining: ₹" + balance);  
        }  
    }  
}
```

```
class WithdrawalThread extends Thread {  
    Bank bank;  
    int amount;  
  
    WithdrawalThread(Bank bank, int amount, String name) {  
        super(name);  
        this.bank = bank;  
        this.amount = amount;  
    }  
  
    public void run() {  
        bank.withdraw(amount, Thread.currentThread().getName());  
    }  
}
```



```
public class BankTest {  
    public static void main(String[] args) {  
        Bank bank = new Bank();  
  
        WithdrawalThread t1 = new WithdrawalThread(bank, 700, "ATM-1");  
        WithdrawalThread t2 = new WithdrawalThread(bank, 600, "OnlineBanking");  
  
        t1.start();  
        t2.start();  
    }  
}
```

Output

```
ATM-1 is going to withdraw ₹700  
ATM-1 has completed withdrawal. Remaining Balance: ₹300  
OnlineBanking tried to withdraw ₹600 but insufficient balance. Remaining: ₹300
```

- Without synchronization, both threads could check the balance before either withdraws, leading to **overdraw**.
- With `synchronized`, only **one thread enters** the `withdraw()` method at a time — ensuring **thread safety**.
- The `Thread.sleep(100)` simulates a **delay**, making the issue visible if synchronization is missing.



◆ Goal: Develop a GUI application using **AWT Controls** and handle events

✓ Concept:

- AWT (Abstract Window Toolkit) is Java's original platform-dependent GUI toolkit.
- Controls include **Buttons, TextFields, Labels, etc.**
- Events (like button clicks) are handled by implementing **listeners** such as `ActionListener`.

✓ Problem: Simple AWT GUI — Calculator for Adding Two Numbers

```
import java.awt.*;
import java.awt.event.*;

public class AddTwoNumbersAWT extends Frame implements ActionListener {

    Label l1, l2, l3;
    TextField t1, t2, t3;
    Button b;

    public AddTwoNumbersAWT() {
        // Set layout manager
        setLayout(new FlowLayout());

        // Create components
        l1 = new Label("Number 1:");
        l2 = new Label("Number 2:");
        l3 = new Label("Result:");
```

```
t1 = new TextField(10);  
t2 = new TextField(10);  
t3 = new TextField(10);  
t3.setEditable(false); // result field non-editable  
  
b = new Button("Add");  
  
// Add components to frame  
add(l1); add(t1);  
add(l2); add(t2);  
add(b);  
add(l3); add(t3);  
  
// Register button click event  
b.addActionListener(this);
```

```
// Set frame properties
setTitle("Add Two Numbers");
setSize(300, 200);
setVisible(true);

// Close window event
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent we) {
        System.exit(0);
    }
});
}
```

```
// Handle button click
```

```
public void actionPerformed(ActionEvent ae) {  
    try {  
        int num1 = Integer.parseInt(t1.getText());  
        int num2 = Integer.parseInt(t2.getText());  
        int sum = num1 + num2;  
        t3.setText(String.valueOf(sum));  
    } catch (NumberFormatException e) {  
        t3.setText("Invalid Input");  
    }  
}
```

```
public static void main(String[] args) {  
    new AddTwoNumbersAWT();  
}
```

```
}
```

- Creates a window with labels and text fields for **input and output**.
 - Has an **Add** button.
 - When clicked, it **adds two numbers** entered by the user.
 - Shows the result or an error message if the input is invalid.
-
- `Frame` is the main window.
 - `Label`, `TextField`, and `Button` are AWT controls.
 - `ActionListener` interface detects button click.
 - `WindowAdapter` handles closing the window properly.
 - Exception handling for invalid input (non-numeric).

12

◆ Goal: Develop a GUI application using **Swing Controls** and handle events

- **Swing** is a lightweight, platform-independent GUI toolkit in Java.
- Provides rich components like `JFrame`, `JButton`, `TextField`, `JLabel`.
- Event handling is done by implementing listeners like `ActionListener`.



Problem: Simple Swing GUI — Calculator for Adding Two Numbers

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddTwoNumbersSwing extends JFrame implements ActionListener {

    private JLabel l1, l2, l3;
    private JTextField t1, t2, t3;
    private JButton b;

    public AddTwoNumbersSwing() {
        // Set frame title
        setTitle("Add Two Numbers - Swing");

        // Set layout manager
        setLayout(new FlowLayout());
    }
}
```

```
// Create components

l1 = new JLabel("Number 1:");
l2 = new JLabel("Number 2:");
l3 = new JLabel("Result:");


t1 = new JTextField(10);
t2 = new JTextField(10);
t3 = new JTextField(10);
t3.setEditable(false);


b = new JButton("Add");


// Add components to frame

add(l1); add(t1);
add(l2); add(t2);
add(b);
add(l3); add(t3);
```

```
// Register event listener  
b.addActionListener(this);  
  
// Set default close operation  
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
// Set size and visibility  
setSize(300, 150);  
setVisible(true);  
}
```

```
// Handle button click
@Override
public void actionPerformed(ActionEvent e) {
    try {
        int num1 = Integer.parseInt(t1.getText());
        int num2 = Integer.parseInt(t2.getText());
        int sum = num1 + num2;
        t3.setText(String.valueOf(sum));
    } catch (NumberFormatException ex) {
        t3.setText("Invalid Input");
    }
}

public static void main(String[] args) {
    // Run GUI in Event Dispatch Thread for thread safety
    SwingUtilities.invokeLater(() -> new AddTwoNumbersSwing());
}
}
```

- Creates a Swing window with labels, text fields, and a button.
 - When the **Add** button is clicked, it sums two numbers entered by the user.
 - Displays the result or an error message.
-
- `JFrame` is the main window container.
 - `JLabel`, `TextField`, and `JButton` are Swing controls.
 - Implements `ActionListener` to handle button clicks.
 - `SwingUtilities.invokeLater()` ensures GUI creation runs on the Event Dispatch Thread.
 - Input validation and exception handling included.